

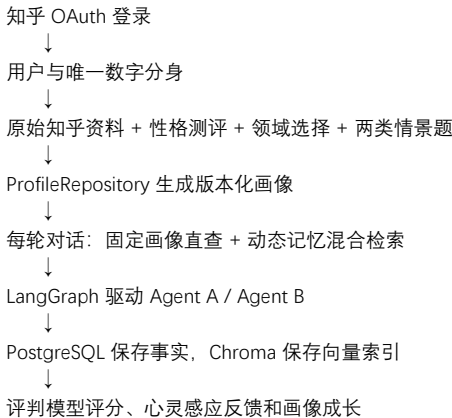
TwinLoop 数字分身项目说明

实现原理、用户体验流程与数据库执行指南 | 2026-09-14

TwinLoop 的目标是：用户用知乎账号登录后，通过知乎内容、性格测评、兴趣和专长选择、观点题与社交情景题，逐步生成一个“有证据、可追溯、会成长”的数字分身；随后让不同用户的数字分身在受控场景中进行对话，并把结果反馈给用户。

奥卡姆剃刀原则：已有模块能完成的能力直接复用，不重复创建第二套知乎客户端、性格评分器、画像仓储或聊天正文表。

1. 系统总体原理



系统分成四层：

- 接入层：知乎 OAuth、会话 Cookie、知乎资料读取。
- 画像层：基础身份、性格、兴趣、专长、观点、行为记忆、表达风格、安全策略和版本。
- 运行层：LangGraph Agent、工具调用、双 Agent 对话、并发限制、评判模型。
- 数据层：PostgreSQL 是事实源；Chroma 只保存向量和最小元数据，不能替代权限和正文。

2. 用户体验流程

阶段一：登录知乎

- 前端调用 POST /api/v1/sources/zhihu/connect。
- 浏览器跳转 authorization_url，用户在知乎确认授权。
- 知乎回调后端 callback；后端校验 state、换取 token、按知乎 uid 建档，并设置 HttpOnly Cookie。
- 前端回到页面后调用 GET /api/v1/me，判断 zhihu_connected 和 avatar_status。

阶段二：初始化数字分身

- 创建初始化会话，记录 import_job_id。
- 读取用户授权范围内的回答、文章和帖子，并保存原始资料。

- 展示 Big Five 性格测评；用户可完整作答或跳过。
- 用户选择兴趣和擅长领域；后端通过领域目录保证 ID 合法。
- 根据领域读取知乎专业问题和争议问题，生成观点选择题。
- 展示限时现实社交情景题，记录选项和耗时。
- 用户填写基础身份，系统把上述结果转换为画像候选，创建第一个 active 版本。

阶段三：进入 Agent 社交场景

- 用户选择场景和匹配方式：LLM 匹配、随机匹配或手动指定。
- 发起方 Agent A 与被邀请方 Agent B 绑定到同一个 conversation_id，但各自使用自己的 avatar_id 和上下文。
- 每轮回答前重新检索，确保新增行为记忆可以立即生效。
- 若没有共同观点，A 随机抛出可公开话题；B 先判断兴趣，低于阈值就自然拒绝并结束。
- 对话消息写入 PostgreSQL，之后尽力写入 Chroma。

阶段四：评判与成长

- 对话结束后，独立评判模型从性格匹配、兴趣激发、观点契合和沟通质量等维度评分。
- 达到配置阈值时只调用推送接口占位，不直接发送真实消息。
- 没有记忆命中时生成心灵感应题；用户选择后累计正向或反向证据。
- 同一性格方向达到累计阈值后才调整画像，避免一次选择造成性格震荡。

3. 画像结构和数据职责

画像部分	回答什么问题	主要来源
profile_identity	用户是谁？	用户填写、知乎公开资料
personality	用户通常有什么稳定性格倾向？	IPIP Big Five 测评和成长反馈
style / style_examples	应该怎样表达？	知乎原文、测评结果、LLM 提炼
expertise / interests	用户擅长和关注什么？	用户选择、知乎内容、LLM 提炼
opinions	用户对某主题怎么看？	观点题、知乎内容、对话反馈
behavior_memories	遇到具体情境时可能怎么反应？	社交情景题、行为记忆工具、对话反馈
source_documents / memory_evidence	依据是什么？	知乎原始内容和引用片段
memory_rules / policy	检索和安全边界是什么？	系统默认策略和用户配置
growth_evaluations	画像如何变化？	对话评判和心灵感应题反馈

4. 检索和 Prompt 组装

固定画像不做向量检索，直接读取当前 active

版本：identity、personality、style、policy。它们回答“用户是谁”和“应该怎样表达”。

经历、观点、行为、专长、兴趣、原始资料和聊天记录属于长内容，先做 PostgreSQL 关键词召回，同时用 Chroma 做语义召回；按 ID 合并后回 PostgreSQL 检查状态、权限、隐私和删除标记。

问题
↓

问题分析与改写
 ↓
 并行：关键词检索 + Chroma 向量检索
 ↓
 按 ID 合并、去重、排序、冲突处理、Token 截断
 ↓
 固定画像 + 动态记忆 + 原始证据 + 当前聊天历史
 ↓
 调用当前 Agent 的 LLM

重要边界：retrieval/service.py 只提供显式调用的函数，不注册为 Agent Tool；Agent 不能自己发现数据库或 Chroma。行为记忆工具只负责提交提案，最终写入仍经过 ProfileRepository。

5. LangGraph 对话原理

单 Agent 图的核心节点是 prepare_context、call_model、execute_tools 和 persist_answer。工具调用完成后回到 prepare_context，重新检索最新画像。双 Agent 编排器负责轮换 A/B，不把两人的画像混在一起。

话题门控：
 load_opinions
 → find_shared_topics
 → 共同观点存在？
 是：select_shared_topic
 否：select_random_topic
 → ask_interest (B 判断兴趣)
 感兴趣：进入正常对话
 不感兴趣：persist_refusal → completed

正常对话：
 A 开场 → B 回答 → B 提问 → A 回答 → ...
 最多 10 轮（最多 20 条 Agent 消息）

当前默认门控阈值：观点相似度 0.65、观点置信度 0.60、B 兴趣分数 0.55。阈值来自环境变量，前端不需要参与计算。

6. PostgreSQL、Chroma 与聊天持久化

- PostgreSQL 保存用户、画像版本、完整知乎原文、完整聊天内容、权限、状态、审计和评判结果。
- Chroma 集合 avatar_memories、source_documents、chat_messages 保存 embedding 和最小 payload。
- 双写顺序是 PostgreSQL 提交成功后再写 Chroma；Chroma 失败不回滚聊天事实，交给 vector_sync_outbox 重试。
- 一次 run_agent_dialogue() 对应一个 agent_chat_groups.chat_no；chat_messages 保存真实正文，查询聊天组时按 sequence_no 升序返回。
- A 和 B 共享 conversation_id，消息发送者由 chat_participants 和 participant_id 识别。

7. SQL 脚本执行顺序

当前仓库存在两套数据结构，执行前必须先确认使用哪一套。

数据结构	特点	适用代码
schema.sql + migrations/002、003	旧原型结构，用户和头像 ID 是 TEXT，database.initialize() 自动执行。	app/main.py 的早期内存画像、性格和领域演示接口。
postgresql_zhihu_schema.sql 等	正式数字分身结构，用户、avatar、聊天和画像 ID 是 UUID，脚本需要手动按依赖执行。	ProfileRepository、retrieval、ChatRepository、Agent 对话模块。

正式 PostgreSQL 数字分身结构的建议执行顺序：

1. postgresql_zhihu_schema.sql

2. postgresql_chat_schema.sql
3. postgresql_profile_outbox.sql
4. postgresql_agent_dialogue.sql
5. postgresql_twin_initialization.sql
6. postgresql_agent_chat_groups.sql

- 第1步创建 users、user_avatars、avatar_versions、avatar_memories、source_documents 等基础表。
- 第2步创建 chat_conversations、chat_participants、chat_messages。
- 第3步创建 PostgreSQL 到 Chroma 的 vector_sync_outbox。
- 第4步创建 agent_dialogue_runs 和 agent_dialogue_evaluations。
- 第5步创建初始化会话、心灵感应反馈和性格调整表。
- 第6步创建面向用户查询的 agent_chat_groups 索引表。

不要混用：postgresql * 脚本依赖 UUID 版 users/user_avatars；schema.sql 是旧 TEXT 版结构。若要让 production runtime 全部使用正式表，应统一数据库连接、用户 ID 类型和迁移入口。

8. 安全、权限和失败处理

- 知乎 token 只保存在服务端会话，日志和 API 响应不返回 API Key 或 access token。
- 敏感、拒绝、过期、软删除或不属于当前 avatar 的记忆不能进入 Prompt。
- 聊天详情必须满足当前用户是 initiator_user_id 或 invited_user_id；否则统一 404。
- 模型失败时对话标记 failed；工具失败允许当前轮继续，但不能伪造“记忆已保存”。
- 评判模型失败时保留聊天记录，禁止触发高质量推送。
- MAX_AGENT_DIALOGUES 默认10，单个双 Agent 对话最多10轮，并用 finally 释放信号量。

9. 前端落地建议

- 登录页：先 connect，再等待 callback，最后请求 /api/v1/me。
- 初始化页：使用一个步骤状态机，页面刷新后通过 GET /v1/twin/initializations/{id} 恢复。
- 领域页：先 GET /v1/domains，再 PUT /v1/domains/selections/me；保存 revision。
- 性格页：先 GET questions，提交时保留 request_key，防止重复提交。
- 聊天历史页：先请求 count 或列表，再点击 chat_no 请求 messages；不要直接拼接 conversation_id 查询。
- 所有长文本展示都应支持加载态、授权过期、可重试错误和空状态。

10. 当前版本的已知边界

- 双 Agent 对话的核心 Python 编排和聊天组查询已存在，但当前没有完整的 HTTP 对话启动、后台任务和取消路由。
- 初始化接口已能保存步骤并创建初始画像，但知乎资料到画像候选的完整 LLM 提炼闭环仍需继续接入。
- runtime 部分接口使用 X-User-Id，正式环境应统一切换为登录依赖，补充 session 到 user_id 的服务端映射。
- LangGraph PostgreSQL Checkpointer 已有适配器文件，但图编译和生命周期仍应在生产启动流程中统一接入。
- 数据库脚本和旧 migration 并行存在，后续应选择一个正式迁移入口，避免两套 ID 类型长期并存。